Deploying DEXter on a DigitalOcean Droplet

This guide takes a fresh Ubuntu 24.04 droplet to a DEXter server answering `https://dexter.example.com/DEXter/pokemon/identify`. It assumes you can use a terminal and have a domain whose DNS you control. Replace `dexter.example.com` with your hostname throughout.

Files in this folder and where they go on the droplet:

File	Installed as
`systemd/dexter.service`	`/etc/systemd/system/dexter.service`
`nginx/dexter.conf`	`/etc/nginx/sites-available/dexter`

1. Create the droplet

In the DigitalOcean control panel choose:

- Image: Ubuntu 24.04 LTS
- Size: 2 GB RAM is recommended, though you can use a 1 GB droplet for testing or alpha with a swap file. The ONNX models and the Pokemon catalog need about 1 GB at runtime, and 2 vCPUs keep identification under half a second.
- Authentication: SSH key
- Hostname: `dexter.example.com`

The hostname gets its DNS record in step 10, just before HTTPS.

2. Packages and firewall

Log in as root and install what the rest of the guide needs.

```
apt update && apt upgrade -y
apt install -y nginx certbot python3-certbot-nginx git ufw curl
ufw allow OpenSSH
ufw allow 'Nginx Full'
ufw --force enable
```

Expect `ufw status` to list `OpenSSH` and `Nginx Full` as allowed.

3. Swap file (1 GB droplets)

Skip this step on a droplet with 2 GB RAM or more. On 1 GB, DEXter's models and catalog do not fit beside the operating system, so the kernel kills the service on its first start. A 2 GB swap file gives it room. Identification is still done in RAM once the service is warm; swap mostly holds parts of the catalog that are rarely touched, so requests stay fast enough for a small number of users.

```
fallocate -l 2G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
grep -q '^/swapfile' /etc/fstab || echo '/swapfile none swap sw 0 0' >> /etc/fstab
echo 'vm.swappiness=10' > /etc/sysctl.d/99-swap.conf
sysctl -p /etc/sysctl.d/99-swap.conf
```

`vm.swappiness=10` tells the kernel to use the swap file only when it has to, instead of moving memory out early. The `fstab` line turns the swap on again after a reboot.

Check it:

```
swapon --show
free -h
```

Expect `swapon --show` to list `/swapfile` at 2G and `free -h` to show a `Swap:` line with that total. If `fallocate` fails with "Operation not supported", use `dd if=/dev/zero of=/swapfile bs=1M count=2048 status=progress` instead and continue from `chmod`.

Also set `DEXTER_NUM_THREADS` to the droplet's vCPU count in step 8, usually `1`; more threads than cores only add memory.

4. A service user and folders

DEXter runs as an unprivileged user that owns only its code and cache.

```
adduser --system --group --home /opt/dexter --shell /usr/sbin/nologin dexter
mkdir -p /var/lib/dexter/cache
chown -R dexter:dexter /opt/dexter /var/lib/dexter
```

5. Install uv and Python

`uv` manages the Python version and the virtual environment. Install it for the service user so the venv is owned by that user.

Each command below starts a shell as `dexter` and changes folder inside it. uv reads settings from the folder it runs in, and `sudo` otherwise keeps root's folder, `/root`, which `dexter` cannot open. uv then stops with `failed to open file `/root/uv.toml`: Permission denied`. `-H` sets the shell's home to `/opt/dexter`, so `cd` alone goes there and `~` stands for it. Later steps run uv and git as `dexter` the same way.

```
sudo -u dexter -H sh -c 'cd && curl -LsSf https://astral.sh/uv/install.sh | sh'
sudo -u dexter -H sh -c 'cd && ~/.local/bin/uv python install 3.12'
```

Expect the second command to print the installed CPython version.

6. SSH key for GitHub

The droplet clones and updates DEXter over SSH, so it needs its own key. The key belongs to the `dexter` user and has no passphrase, so updates can run unattended:

```
sudo -u dexter mkdir -p -m 700 /opt/dexter/.ssh
sudo -u dexter ssh-keygen -t ed25519 -N "" -f /opt/dexter/.ssh/id_ed25519
sudo -u dexter sh -c 'ssh-keyscan -t ed25519 github.com >> /opt/dexter/.ssh/known_hosts'
cat /opt/dexter/.ssh/id_ed25519.pub
```

Copy the line the last command prints; it starts with `ssh-ed25519`. On GitHub, open the repository's **Settings > Deploy keys > Add deploy key**, paste it, and give it a title such as `dexter droplet`. Leave **Allow write access** off: the droplet only reads. Then check it:

```
sudo -u dexter ssh -T git@github.com
```

`Hi henshinger555/DEXter! You've successfully authenticated, but GitHub does not provide shell access.` means it works. `Permission denied (publickey)` means the key is not on the repository yet.

Only the repository's owner can add a deploy key. The repository is public, so if you are deploying it without that access, skip this step and clone over HTTPS in step 7 instead: `https://github.com/henshinger555/DEXter.git`.

7. The code

```
sudo -u dexter -H sh -c 'cd && git clone git@github.com:henshinger555/DEXter.git app'
sudo -u dexter -H sh -c 'cd ~/app && ~/.local/bin/uv sync --no-dev'
```

The clone checks out the default branch, `main`, which is what a server runs. `uv sync` creates `/opt/dexter/app/.venv`, installs CollectorVision from GitHub at the commit pinned in `pyproject.toml`, and installs the `dexter` command. It takes a minute or two; ONNX Runtime and OpenCV are the large downloads.

8. Configuration

```
cp /opt/dexter/app/.env.example /opt/dexter/app/.env
chown dexter:dexter /opt/dexter/app/.env
chmod 600 /opt/dexter/app/.env
nano /opt/dexter/app/.env
```

Set at least:

```
DEXTER_GAMES=pokemon
DEXTER_CACHE_DIR=/var/lib/dexter/cache
DEXTER_HOST=127.0.0.1
DEXTER_PORT=8000
DEXTER_CORS_ORIGINS=https://app.example.com
```

`DEXTER_CORS_ORIGINS` lists the browser origins allowed to call the API. Leave it empty if only servers will call DEXter. On a 1 GB droplet also set `DEXTER_NUM_THREADS=1` (or your vCPU count).

Do a first run by hand so the model and catalog downloads happen where you can watch them:

```
sudo -u dexter -H sh -c 'cd ~/app && .venv/bin/dexter'
```

Expect a line like `Loaded pokemon CatalogV2(... n=27275) with NeuralEmbedder(...)` followed by `Uvicorn running on http://127.0.0.1:8000`. Press Ctrl+C.

If the download fails, check that the droplet can reach `https://huggingface.co` and that `/var/lib/dexter/cache` is writable by `dexter`.

9. The systemd service

The unit file in this folder uses the paths above. If you changed any, edit the copy before enabling it.

```
cp /opt/dexter/app/deploy/systemd/dexter.service /etc/systemd/system/dexter.service
systemctl daemon-reload
systemctl enable --now dexter
systemctl status dexter --no-pager
curl http://127.0.0.1:8000/DEXter/health
```

Expect `active (running)` and `{"status":"ok","games":["pokemon"],...}`.

`TimeoutStartSec=300` in the unit allows for a slow first download; after the cache is warm the service starts in a few seconds.

Logs: `journalctl -u dexter -f`.

10. DNS record in Cloudflare

certbot in the next step proves you control the hostname by fetching a file from it over plain HTTP, so the name must already lead to this droplet. DEXter gets a subdomain of your site's domain, such as `dexter.example.com`, pointing at this droplet, while `example.com` keeps pointing at the site's own droplet. Browsers connect to whichever address the name resolves to; nothing on the site's droplet forwards to DEXter.

The droplet's addresses

On the droplet, ask DigitalOcean's metadata service, which only answers from inside the droplet:

```
curl -s http://169.254.169.254/metadata/v1/interfaces/public/0/ipv4/address; echo
curl -s http://169.254.169.254/metadata/v1/interfaces/public/0/ipv6/address; echo
```

The first line is the public IPv4 address. The second is the IPv6 address if you enabled IPv6 on the droplet; if it prints no address, skip the `AAAA` record below. Both also appear at the top of the droplet's page in the DigitalOcean control panel.

Add the record

1. Sign in at https://dash.cloudflare.com and choose your account.
2. Under **Domains**, select your domain, for example `example.com`. Its status must be **Active**. **Pending** means the nameservers at your registrar do not point at Cloudflare yet, and records you add here are not published.
3. In the left sidebar open **DNS > Records**.
4. Look for an existing record named `dexter`. If there is one, edit it rather than adding another: Cloudflare will not save an `A` record next to a `CNAME` of the same name.
5. Select **Add record** and fill it in:

Field	Value
Type	`A`
Name	`dexter`: only the part before your domain. Cloudflare adds `.example.com` and shows the full name under the field.
IPv4 address	the first address from above
Proxy status	switched off, so the cloud is grey and reads **DNS only**
TTL	**Auto**

6. Select **Save**.
7. If the droplet has IPv6, select **Add record** again with Type `AAAA`, the same Name, the IPv6 address and **DNS only**, then **Save**.

Leave the records on **DNS only** for now. Proxied (orange cloud) makes the name resolve to Cloudflare's addresses instead of the droplet's, so certbot and the checks below would test Cloudflare and not your server. "Optional: Cloudflare's proxy" after step 12 turns it on once everything works.

If your domain's DNS is not at Cloudflare, add the same `A` record at whichever provider holds it.

Check it

From your own computer or the droplet:

```
dig +short dexter.example.com @1.1.1.1
```

On Windows use `nslookup dexter.example.com 1.1.1.1`. Expect the droplet's IPv4 address. Cloudflare publishes a new record within a minute or two.

Result	Meaning
The droplet's address	Done. Continue.
Nothing, or "can't find" / NXDOMAIN	Not published yet, or the Name is misspelled. If you looked the name up before creating it, resolvers remember the "no such name" answer for up to 30 minutes. Ask Cloudflare's own nameservers, which never cache it: they are listed under **Cloudflare Nameservers** on the **DNS > Records** page, for example `dig +short dexter.example.com @ada.ns.cloudflare.com`.
An address starting with `104.`, `172.64.` to `172.71.`, or `2606:4700:`	The record is Proxied. Edit it and switch the cloud to grey.
Some other address	The wrong IP was pasted, or an older record still holds the name. Edit it.

Last, check that the droplet answers on that name over HTTP, as certbot will. From your own computer (on Windows type `curl.exe`):

```
curl -sI http://dexter.example.com
```

Expect `HTTP/1.1 200 OK` and a `Server:` line naming nginx: its welcome page from step 2, which is still enabled. A timeout means the firewall is not letting port 80 in (`ufw status` must list `Nginx Full`) or the record points somewhere else.

11. nginx and HTTPS

Continue once the checks in step 10 pass. nginx terminates TLS and forwards `/DEXter/` to the service. Everything else returns 404.

```
cp /opt/dexter/app/deploy/nginx/dexter.conf /etc/nginx/sites-available/dexter
sed -i 's/dexter.example.com/YOUR-HOSTNAME/' /etc/nginx/sites-available/dexter
ln -s /etc/nginx/sites-available/dexter /etc/nginx/sites-enabled/dexter
rm -f /etc/nginx/sites-enabled/default
nginx -t && systemctl reload nginx
certbot --nginx -d YOUR-HOSTNAME --agree-tos -m you@example.com --redirect
```

certbot rewrites the site file with the certificate paths and an HTTP to HTTPS redirect, and installs a timer that renews it. Check with `certbot renew --dry-run`.

`client_max_body_size 11m` in the nginx file must stay at or above `DEXTER_MAX_UPLOAD_BYTES`, otherwise nginx answers 413 before DEXter sees the request.

12. Check from outside

From your own machine:

```
curl https://YOUR-HOSTNAME/DEXter/health
curl -F "file=@card.jpg" https://YOUR-HOSTNAME/DEXter/pokemon/identify
```

The first returns the health JSON. The second returns `card_present: true` with a `match` for a clear photo of a Pokemon card on a plain background, or `card_present: false` with a low `sharpness` if the detector could not find a card. Interactive docs are at `https://YOUR-HOSTNAME/DEXter/docs`.

Optional: Cloudflare's proxy

Once step 12 passes you can switch the record from step 10 to **Proxied** (orange cloud) in **DNS > Records**, so the droplet's address is hidden and Cloudflare absorbs floods before they reach it. The free plan's certificate covers one level of subdomain, so `dexter.example.com` is included. First set **SSL/TLS > Overview** to **Full (strict)**: Cloudflare then talks to nginx over https and checks the certbot certificate. Never use Flexible: Cloudflare would ask nginx over http, nginx redirects to https, and every request loops.

Two things change behind the proxy:

- nginx sees Cloudflare's addresses, not the caller's. If that matters for your logs or limits, configure nginx's real-ip module with the ranges from https://www.cloudflare.com/ips/ and the `CF-Connecting-IP` header.
- Uploads pass through Cloudflare, whose free-plan request limit of 100 MB is far above `DEXTER_MAX_UPLOAD_BYTES`. Nothing to change.

If identification stops working right after the switch, set the record back to DNS only and retest. A working site on DNS only and a broken one on Proxied means a Cloudflare setting, almost always the SSL mode.

Updating

```
sudo -u dexter -H sh -c 'cd ~/app && git pull && ~/.local/bin/uv sync --no-dev'
systemctl restart dexter
curl http://127.0.0.1:8000/DEXter/health
```

If a release changes the CollectorVision pin, the new catalog downloads on the next start. To go back to an earlier release:

```
sudo -u dexter -H sh -c 'cd ~/app && git checkout <previous-tag>'
sudo -u dexter -H sh -c 'cd ~/app && ~/.local/bin/uv sync --no-dev'
systemctl restart dexter
```

Catalogs update themselves: CollectorVision checks its feed on each start and downloads a newer snapshot when one exists. Restart the service occasionally to pick up new card printings.

Adding a game

Edit `/opt/dexter/app/.env`, append the game to `DEXTER_GAMES` (for example `DEXTER_GAMES=pokemon,pokemon-japan`), and `systemctl restart dexter`. The game must exist in `src/dexter/games.py`; see the root README for adding new ones.

Troubleshooting

Symptom	Likely cause
`systemctl status dexter` shows `failed` within seconds	A `.env` value is invalid or `DEXTER_GAMES` names an unknown game. `journalctl -u dexter` shows the error.
Service hangs in `activating`	First-run download is slow. Wait up to five minutes, or run the manual start from step 8 to see progress.
certbot fails with `NXDOMAIN`, `Timeout during connect` or `unauthorized`	The name does not lead to this droplet over HTTP yet. Rerun the checks in step 10; the record must be DNS only and point at this droplet.
`502 Bad Gateway` from nginx	The service is not listening on `127.0.0.1:8000`. Check `DEXTER_HOST` / `DEXTER_PORT` against the nginx `proxy_pass`.
`413` on every upload	`client_max_body_size` in nginx is below the image size.
`card_present: false` for real cards	The photo is blurry or the card does not fill enough of the frame. Try `DEXTER_MIN_SHARPNESS=0` to see the raw sharpness value, then pick a threshold.
Memory errors in the journal, or the service is killed during startup	Droplet has under 2 GB RAM and no swap. Add the swap file from step 3, or resize the droplet.
Requests take several seconds on a 1 GB droplet	Swap is doing the work of RAM. Check `free -h`; if `Swap:` used is well over half, resize the droplet.

Backups

Nothing on the droplet is unique. The code is in git, the models and catalog are re-downloaded into `DEXTER_CACHE_DIR`, and `.env` is the only file worth copying somewhere safe.